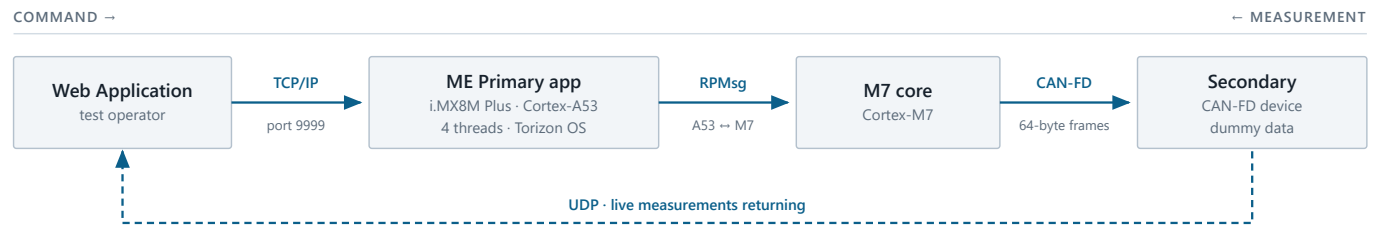


The ME Primary Demonstration

On 21 August 2026 the ME Primary Board completed an **end-to-end run of the full battery-test data path** — from channel registration on the Web Application, through program delivery and execution on the i.MX8M Plus, out over CAN-FD to the Secondary, and back again as live telemetry. The application that carries this out was built in about three weeks with AI assistance. This paper records what was demonstrated and the working method that made it repeatable.

THE PATH THAT WAS DEMONSTRATED



The complete round trip carried out in the demonstration. Every part of this path ran for real. Only the Secondary was substituted, returning dummy values in place of hardware that has not yet been built.

WHAT RAN, IN ORDER

- 01

Channel registration. Each test channel identifies itself to the Web Application over TCP. It must be accepted before it is allowed to carry any traffic.
- 02

Program setup. The operator creates a battery-test program in the Web Application, defining its steps, its limits and the conditions that end the test.
- 03

Program delivery. The program reaches the board over Ethernet as 0xBB packets. Each packet is checked with a CRC and decoded into a sequence of steps the board can run.
- 04

Hand-off between threads. Four threads — Communication, Core Logic, Data Manager and CAN Manager — each handle one job and pass the work between them using internal message queues.
- 05

Execution on START. A 0xEE control command starts the step engine. It applies the setpoints for each step in turn and checks them against the programmed limits.
- 06

CAN-FD output through the M7. The setpoints are passed to the Cortex-M7 core over RPMsg and sent onto the bus as 64-byte CAN-FD frames addressed to the Secondary.
- 07

Measurements returned. The frames coming back are decoded on the A53 into voltage and current readings for each channel.
- 08

Live telemetry. Those readings are sent to the Web Application as 0xCC real-time packets over UDP, which completes the loop.

THE APPLICATION, IN NUMBERS

8,327	4,942	229	13	39
lines of production C across 62 files	lines of test code, 18 suites	automated checks — 0 failing	design & plan documents written before code	architecture decisions recorded with reasons

SCOPE OF THE DEMONSTRATION

PROVEN ON HARDWARE	SUBSTITUTED	STILL TO COME
Registration, program delivery and decoding, execution across the four threads, the A53–M7 RPMsg link, CAN-FD transmission, and live telemetry to the Web Application.	The Secondary board has not been built yet, so the M7 returned dummy values in its place. This proves the data path and the decoding, but not the analogue measurement itself.	A full battery-test run against a real Secondary, and automatic detection of which channels are physically present. This has been designed and is awaiting review.

The engineering judgement did not come from the AI. Four things were already in place before any code was written, and one working rule was followed throughout. The four gave the AI a correct foundation to build on. The rule is the reason it built the right thing the first time.

KNOWLEDGE OF THE PRODUCT

More than a year of work on a similar battery-test product meant that the required behaviour was already clear. No requirement had to be settled during the build, and any incorrect suggestion was identified immediately.

THE EARLIER BTS CODE

Code from the earlier BTS system, written manually by the team, was used as a reference for the program logic and for the behaviour of each test step. Proven behaviour was reused rather than designed again from scratch.

PROTOCOL DOCUMENTS WRITTEN BEFOREHAND

Nine specifications were written before coding began. They covered the TCP and UDP packet layouts, registration, control, configuration, and how a test program is encoded and decoded. Every frame format was taken from a document rather than guessed.

THE DIGATRON CAN-FD SPECIFICATION

The master–slave CAN-FD document from the Digatron team defined exactly how the Primary and the Secondary exchange data. Because that format was already fixed, the task was to implement it rather than to design it.

THE METHOD THAT MADE IT REPEATABLE

APPLIED STRICTLY TO EVERY FEATURE FOR THREE WEEKS

Write the design first. Approve it. Then implement it.

Context and specs given



AI drafts the design



Engineer approves



AI implements

No feature, however small, was implemented before its design document had been read and approved. Any disagreement was settled on the document, where a correction costs one sentence, instead of in the code, where it costs a rewrite. Thirteen design and plan documents were approved in this way before their code was written, and thirty-nine architecture decisions were recorded together with the reasons for them. The result was a build that needed very few retries or corrections, because the intent was agreed before any code existed and the code then matched it.

WHAT THIS MEANS FOR FUTURE WORK

The speed came from having clear limits, not from working without them. The AI was fast because the target was never in doubt: complete protocol documents, a working reference implementation, an engineer who already knew the correct answer, and an approved design before every change. With the same conditions in place, this method should work equally well on the other boards in this family. Without them — unclear requirements, undocumented protocols, or no review step — the same tools would still produce code that looks correct but is not. The review effort would not disappear. It would simply move from reading a document to debugging the board.